

Systeme d'analyse hospitalière multi-agents

Durée : 2h - 2h30
Documents autorisés : fichiers fournis uniquement

Contexte

Vous êtes développeur dans un hôpital. Le directeur médical souhaite interroger les données patients en langage naturel et obtenir des analyses structurées avec alertes automatiques.

Vous disposez d'une architecture multi-agents a compléter et enrichir.

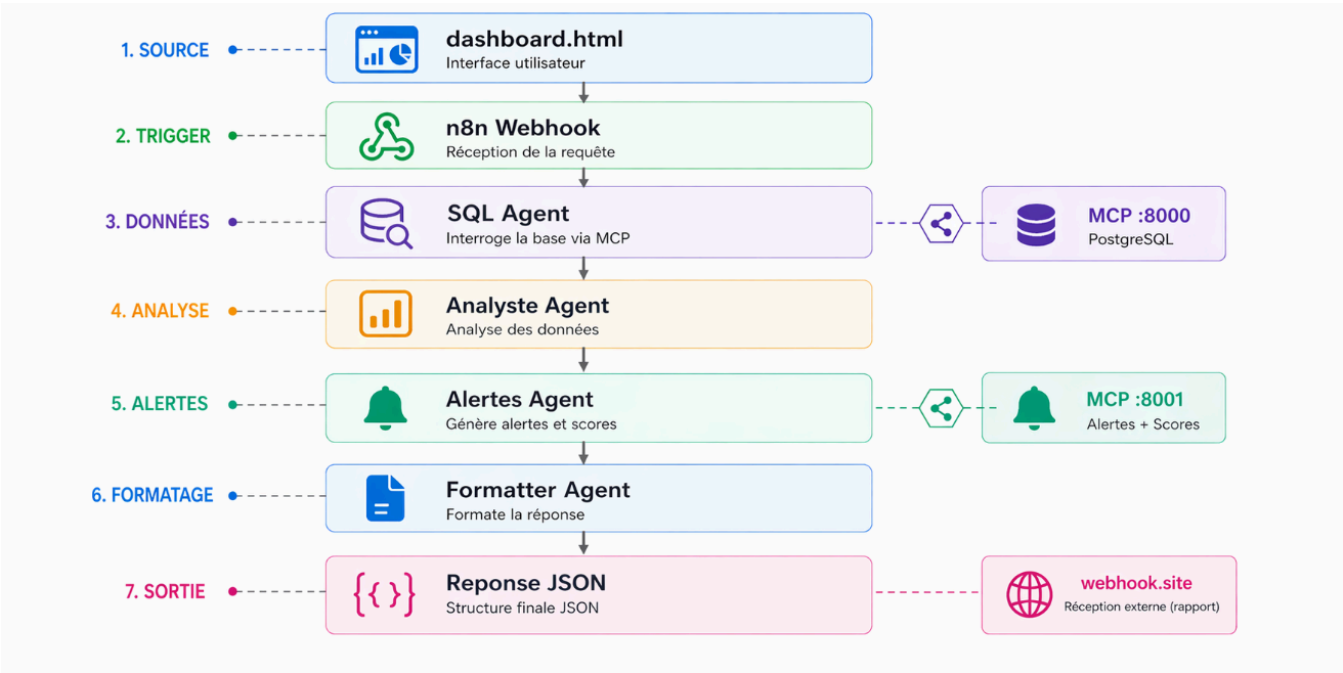
Fichiers fournis

Les fichiers suivants sont disponibles dans le dossier de départ :

- `init_db.sql` — script de création de la base PostgreSQL
- `mcp_server.py` — serveur MCP PostgreSQL (port 8000)
- `mcp_alertes.py` — serveur MCP alertes medicales (port 8001)
- `dashboard.html` — interface Chart.js

Le contenu de chaque fichier est fourni en annexe a la fin de ce document.

Architecture cible



Partie 1 — Base de données

Vous utilisez Supabase comme base de données PostgreSQL. Aucune installation locale n'est nécessaire.

Etape 1.1 — Accéder a éditeur SQL

Connectez-vous sur votre projet Supabase : <https://supabase.com/dashboard>

Dans le menu gauche, cliquez sur **SQL Editor**.

Etape 1.2 — Exécuter le script

Copiez intégralité du contenu de `init_db.sql` fourni en annexe et collez-le dans éditeur SQL. Cliquez sur **Run**.

Le script crée les tables et insère les données directement dans votre base Supabase.

Etape 1.3 — Verification

Toujours dans le SQL Editor, exécutez les trois requêtes suivantes une par une :

```
SELECT COUNT(*) FROM patients;
SELECT COUNT(*) FROM sejours;
SELECT COUNT(*) FROM diagnostics;
```

Resultats attendus : 25 lignes dans chaque table.

Etape 1.4 — Recuperer les parametres de connexion

Dans le menu gauche, cliquez sur **Project Settings** puis **Database**.

Notez les informations suivantes, vous en aurez besoin pour configurer `mcp_server.py` :

Parametre	Ou le trouver
<code>host</code>	Host (section Connection parameters)
<code>port</code>	5432 (valeur par default)
<code>database</code>	postgres
<code>user</code>	postgres
<code>password</code>	Le mot de passe defini a la creation du projet

Ces valeurs sont a renseigner dans le bloc `DB_CONFIG` de `mcp_server.py`.

Partie 2 — Serveurs MCP

Installation des dépendances

```
pip install fastapi uvicorn psycpg2-binary mcp
```

Configuration

Ouvrez `mcp_server.py` et adaptez le bloc `DB_CONFIG` avec vos parametres PostgreSQL :

```
DB_CONFIG = {
    "host":      "localhost",
    "port":      5432,
    "database":  "hopitaldb",
    "user":      "...",
    "password":  "..."
}
```

Lancement

Ouvrez deux terminaux et lancez les deux serveurs :

```
# Terminal 1
python mcp_server.py

# Terminal 2
python mcp_alertes.py
```

Verification

```
curl http://127.0.0.1:8000/mcp
curl http://127.0.0.1:8001/mcp
```

Partie 3 — Pipeline n8n

Créez un workflow n8n nommé `Hopital Multi-Agents`.

Node 1 — Webhook

Champ	Valeur
Method	POST
Path	hopital
Respond	Last Node

Node 2 — SQL Agent

Connectez ce node au Webhook.

Paramètres :

Champ	Valeur
Prompt Type	Delfine
Modele	gpt-4o-mini
Memoire	aucune
Tools	MCP Client http://127.0.0.1:8000/mcp

User Message :

```
Question: {{ $json.body.query }}
```

Rédigez le system message de cet agent. Il doit :

- appeler `get_schema()` en premier
- generer une requete SQL avec les jointures necessaires
- appeler `run_sql()` et retourner les donnees brutes

Test intermédiaire — SQL Agent seul

Dans n8n, cliquez sur le noeud **Webhook** puis sur **Listen for test event**. Le noeud passe en attente (icône orange).

Ouvrez un terminal et lancez :

```
curl -X POST http://localhost:5678/webhook-test/hopital \
-H "Content-Type: application/json" \
-d '{"query": "Combien y a-t-il de patients dans chaque service ?"}'
```

Dans l'interface n8n, observez l'exécution. Le SQL Agent doit retourner un tableau JSON brut. Par exemple :

```
[
  { "service": "Cardiologie", "total_patients": 6 },
  { "service": "Chirurgie", "total_patients": 5 },
  { "service": "Neurologie", "total_patients": 4 },
  { "service": "Urgences", "total_patients": 5 },
  { "service": "Pediatrie", "total_patients": 4 }
]
```

Points à vérifier :

- `get_schema()` apparait dans les logs avant `run_sql()`
- Les données correspondent aux 25 patients insérés en Partie 1

- Aucune erreur de connexion MCP

Si vous obtenez une erreur de connexion MCP, vérifiez que `mcp_server.py` tourne bien dans un terminal actif.

Node 3 — Analyste Agent

Connectez ce node au SQL Agent.

Paramètres :

Champ	Valeur
Prompt Type	Define
Modele	gpt-4o-mini
Memoire	aucune

User Message :

```
Question : {{ $('webhook').item.json.body.query }}
Donnees  : {{ $json.output }}
```

Redigez le system message de cet agent. Il doit retourner uniquement un JSON valide avec la structure suivante :

```
{
  "summary": "...",
  "metrics": {
    "total_patients": 0,
    "age_moyen": 0,
    "service_plus_charge": "...",
    "cout_total": 0
  },
  "insights": [...],
  "anomalies": [...],
  "charts": {
    "patients_par_service": [{ "service": "...", "value": 0 }],
    "diagnostics_frequents": [{ "diagnostic": "...", "total": 0 }]
  },
  "conclusion": "..."
}
```

Test intermédiaire — Analyste Agent seul

Dans n8n, cliquez sur le noeud **Webhook** puis sur **Listen for test event**. Le noeud passe en attente (icône orange).

Ouvrez un terminal et lancez :

```
curl -X POST http://localhost:5678/webhook-test/hopital \
-H "Content-Type: application/json" \
-d '{"query": "Combien y a-t-il de patients dans chaque service ?"}'
```

Dans l'interface n8n, observez la sortie du noeud Analyste Agent. Elle doit être un JSON valide avec cette structure :

```
{
  "summary": "...",
  "metrics": {
    "total_patients": 25,
    "age_moyen": 67.2,
    "service_plus_charge": "Cardiologie",
    "cout_total": 82750.00
  },
  "insights": [
    "La Cardiologie concentre le plus grand nombre de patients",
    "..."
  ],
  "anomalies": [...],
  "charts": {
    "patients_par_service": [{ "service": "Cardiologie", "value": 6 }],
    "diagnostics_frequents": [{ "diagnostic": "...", "total": 0 }]
  },
  "conclusion": "..."
}
```

Points à vérifier :

- La sortie est un JSON pur, sans backticks ni texte autour
- Toutes les clés `summary`, `metrics`, `insights`, `anomalies`, `charts`, `conclusion` sont présentes
- Les valeurs de `metrics` correspondent aux données réelles de la base
- La référence `{{ $('webhook').item.json.body.query }}` dans le User Message retourne bien la question et non une valeur vide — si c'est le cas, vérifiez que le noeud Webhook est bien nommé `webhook` dans n8n

Si le JSON contient des backticks ou du texte parasite, renforcez le system message avec la ligne suivante :

Réponds UNIQUEMENT avec le JSON. Aucun texte avant, aucun texte après, aucune balise markdown.

Node 4 — Alertes Agent

Connectez ce node au Comparsateur Agent (voir Partie 4).

Parametres :

Champ	Valeur
Prompt Type	Define

Champ	Valeur
Modele	gpt-4o-mini
Memoire	aucune
Tools	MCP Client http://127.0.0.1:8001/mcp

User Message :

```
Analyse JSON : {{ $json.output }}
```

Rédigez le system message de cet agent. Il doit :

- identifier les situations a risque dans l'analyse
- appeler `creer_alerte()` pour chaque situation critique
- appeler `calculer_score_risque()` si des patients hospitalises sont mentionnes
- appeler `envoyer_rapport()` si au moins une alerte critique est generee
- retourner un JSON avec la liste des alertes creees

Test intermédiaire — Alertes Agent

Dans n8n, cliquez sur le noeud **Webhook** puis sur **Listen for test event**. Le noeud passe en attente (icône orange).

Ouvrez un terminal et lancez :

```
curl -X POST http://localhost:5678/webhook-test/hopital \
-H "Content-Type: application/json" \
-d '{"query": "Quels sont les patients hospitalises avec un diagnostic grave ?"}'
```

Dans l'interface n8n, observez la sortie du noeud Alertes Agent. Elle doit être un JSON avec cette structure :

```
{
  "alertes_creees": [
    {
      "niveau": "critique",
      "service": "Cardiologie",
      "message": "Patient hospitalise avec infarctus aigu du myocarde, gravite 5"
    },
    {
      "niveau": "warning",
      "service": "Neurologie",
      "message": "Patient hospitalise avec syndrome de Guillain-Barre, gravite 4"
    }
  ],
  "rapport_envoye": true
}
```

Points à vérifier :

- Les appels à `creer_alerte()` apparaissent bien dans les logs d'exécution du noeud
- `rapport_envoye` vaut `true` si au moins une alerte de niveau `critique` a été générée
- `rapport_envoye` vaut `false` si toutes les alertes sont de niveau `warning` ou `info`
- Le serveur `mcp_alertes.py` tourne bien sur le port 8001 — si vous obtenez une erreur de connexion MCP, vérifiez que le terminal qui le lance est toujours actif
- `{{ $json.output }}` dans le User Message reçoit bien la sortie du Comparateur et non une valeur vide

Si aucune alerte n'est générée, renforcez le system message de l'Alertes Agent en précisant explicitement le seuil de déclenchement :

Toute situation avec gravite ≥ 4 doit générer une alerte de niveau "critique".
Toute situation avec gravite $= 3$ doit générer une alerte de niveau "warning".

- Si le champ `service` contient `"1"` et `"5"` au lieu du nom du service.

Node 5 — Formatter Agent

Connectez ce node a l'Alertes Agent.

Paramètres :

Champ	Valeur
Prompt Type	Define
Modele	gpt-4o-mini
Memoire	aucune

User Message :

Analyse : `{{ $('Analyste').item.json.output }}`
Alertes : `{{ $json.output }}`

Rédigez le system message de cet agent. Il doit fusionner l'analyse et les alertes en un seul JSON final avec cette structure :

```
{
  "summary": "...",
  "metrics": { "total_patients": 0, "age_moyen": 0, "service_plus_charge": "...",
"cout_total": 0 },
  "insights": [...],
  "anomalies": [...],
  "alertes": [{ "niveau": "...", "service": "...", "message": "..." }],
  "charts": {
    "patients_par_service": [{ "service": "...", "value": 0 }],
    "diagnostics_frequents": [{ "diagnostic": "...", "total": 0 }]
  },
  "conclusion": "..."
}
```

Test intermédiaire — Formatter Agent

Dans n8n, cliquez sur le noeud **Webhook** puis sur **Listen for test event**. Le noeud passe en attente (icône orange).

Ouvrez un terminal et lancez :

```
curl -X POST http://localhost:5678/webhook-test/hopital \
-H "Content-Type: application/json" \
-d '{"query": "Quels sont les patients hospitalises avec un diagnostic grave ?"}'
```

Dans l'interface n8n, observez la sortie du noeud Formatter Agent. Elle doit être un JSON unique fusionnant l'analyse et les alertes :

```
{
  "summary": "...",
  "metrics": {
    "total_patients": 25,
    "age_moyen": 63.3,
    "service_plus_charge": "Cardiologie",
    "cout_total": 0
  },
  "insights": [...],
  "anomalies": [...],
  "alertes": [
    {
      "niveau": "critique",
      "service": "Cardiologie",
      "message": "..."
    }
  ],
  "charts": {
    "patients_par_service": [{ "service": "...", "value": 0 }],
    "diagnostics_frequents": [{ "diagnostic": "...", "total": 0 }]
  },
  "conclusion": "..."
}
```

Points à vérifier :

- Le JSON final contient bien les deux sources fusionnées : les champs de l'Analyste ET le champ `alertes` de l'Alertes Agent
- La clé `alertes` est un tableau et non une chaîne de caractères
- Les références dans le User Message pointent vers les bons nœuds :
 - `{{ $('Analyste Agent').item.json.output }}` — vérifiez que le nom du nœud correspond exactement à celui dans n8n
 - `{{ $.json.output }}` — reçoit bien la sortie de l'Alertes Agent
- Le JSON retourné est pur, sans backticks ni texte parasite

Si le Formater retourne une erreur sur les références de nœuds, vérifiez les noms exacts de chaque nœud dans n8n et ajustez les expressions en conséquence.

Partie 5 — Bloc B : Tool MCP calculateur de criticité

Dans cette partie vous allez ajouter un nouvel outil dans `mcp_alertes.py`. Cet outil calcule un score de criticité patient à partir d'un code diagnostic CIM, d'un âge et d'une durée de séjour, sans aucune interaction avec la base de données.

Code à ajouter dans `mcp_alertes.py`

Ajoutez la fonction suivante avant le bloc `if __name__ == "__main__":` :

```
@mcp.tool()
def calculer_score_criticite(code_cim: str, age: int, duree_sejour: int) -> str:
    """
    Calcule un score de criticite patient sur 100 a partir d un diagnostic CIM,
    d un age et d une duree de sejour. Aucune base de donnees requise.

    code_cim      : code diagnostic CIM (ex: I21.0)
    age           : age du patient en annees
    duree_sejour  : duree du sejour en jours
    """

    # Table de correspondance CIM -> gravite (1 a 5)
    GRAVITES_CIM = {
        "I21": 5, "I21.0": 5, "I21.4": 5,
        "I63": 5, "I63.5": 5,
        "C34": 5, "C34.1": 5,
        "I50": 4, "I50.0": 4,
        "I48": 4,
        "G61": 4, "G61.0": 4,
        "G35": 3,
        "G40": 3, "G40.0": 3,
        "I10": 3,
        "G20": 3,
```

```

        "G43": 2, "G43.0": 2,
        "K57": 3, "K57.3": 3,
        "J18": 3, "J18.9": 3,
        "S06": 2, "S06.0": 2,
        "K40": 2, "K40.0": 2,
        "M16": 2, "M16.0": 2,
        "S52": 2, "S52.0": 2,
        "R07": 2, "R07.4": 2,
        "A09": 1,
        "T14": 1, "T14.1": 1,
        "J06": 1, "J06.9": 1,
        "L03": 1, "L03.0": 1,
        "R55": 1,
        "K80": 1, "K80.0": 1,
    }

    # Recherche de la gravite : code exact puis prefixe
    gravite = GRAVITES_CIM.get(code_cim.upper())
    if gravite is None:
        prefixe = code_cim.split(".")[0].upper()
        gravite = GRAVITES_CIM.get(prefixe, 3) # default : gravite 3

    # Calcul des composantes
    score_age = round(min(age / 100 * 25, 25), 1)
    score_gravite = gravite * 10
    score_duree = round(min(duree_sejour / 30 * 25, 25), 1)
    score_total = round(score_age + score_gravite + score_duree, 1)

    # Recommandation selon le score
    if score_total >= 75:
        recommandation = "Transfert en soins intensifs recommande"
    elif score_total >= 50:
        recommandation = "Surveillance renforcee requise"
    elif score_total >= 25:
        recommandation = "Suivi standard"
    else:
        recommandation = "Sortie envisageable"

    resultat = {
        "code_cim": code_cim,
        "age": age,
        "duree_sejour": duree_sejour,
        "gravite_cim": gravite,
        "score_total": score_total,
        "detail": {
            "score_age": score_age,
            "score_gravite": score_gravite,
            "score_duree": score_duree
        },
        "recommandation": recommandation
    }

    return json.dumps(resultat, ensure_ascii=False)

```

Relancement du serveur

Redémarrez `mcp_alertes.py` pour que le nouvel outil soit pris en compte :

```
python mcp_alertes.py
```

Verification avec MCP Inspector

Ouvrez MCP Inspector sur `http://localhost:6274`, connectez-vous sur `http://127.0.0.1:8001/mcp` et appelez l'outil avec les parametres suivants :

```
{
  "code_cim": "I21.0",
  "age": 72,
  "duree_sejour": 7
}
```

Resultat attendu :

```
{
  "code_cim": "I21.0",
  "age": 72,
  "duree_sejour": 4,
  "gravite_cim": 5,
  "score_total": 79.8,
  "detail": {
    "score_age": 18.0,
    "score_gravite": 50,
    "score_duree": 5.8
  },
  "recommandation": "Transfert en soins intensifs recommande"
}
```

Mise a jour du system message de l'Alertes Agent

Ajoutez l'instruction suivante dans le system message de l'Alertes Agent :

```
Quand un patient hospitalise est mentionne avec son diagnostic et son age,
appelle calculer_score_criticite() avec le code CIM du diagnostic, l'age du
patient et la duree de sejour en jours.
Inclus le score retourne dans le message de l'alerte correspondante.
```

Partie 6 — Bloc C : Envoi du rapport vers webhook.site

Dans cette partie, vous allez ajouter une etape de diffusion externe au pipeline. A chaque analyse, le JSON final sera envoye vers une URL webhook publique, simulant l'envoi d'un rapport vers un systeme tiers.

Etape 6.1 — Generer votre URL webhook

Ouvrez un navigateur et rendez-vous sur :

```
https://webhook.site
```

A l'ouverture, le site genere automatiquement une URL unique de la forme :

```
https://webhook.site/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
```

Copiez cette URL. Laissez l'onglet ouvert : vous verrez les requetes arriver en temps reel.

Etape 6.2 — Ajouter un node HTTP Request dans n8n

Ajoutez un node apres le Formatter Agent.

Champ	Valeur
Method	POST
URL	votre URL webhook.site
Authentication	None
Body Content	JSON

Dans le corps de la requete, transmettez la sortie du Formatter Agent :

```
{{ $json.output }}
```

Si la sortie est deja un objet JSON parse, utilisez plutot :

```
{{ JSON.stringify($json.output) }}
```

Cliquez sur Save.

Etape 6.3 — Configurer la reponse finale du Webhook n8n

Ajoutez un node **Respond to Webhook** apres le HTTP Request.

Champ	Valeur
Respond With	JSON
Body	{{ \$('Formatter Agent').item.json.output }}

Ce node garantit que le client curl recoit toujours le JSON d'analyse propre, independamment de la reponse de webhook.site.

Son ajout impose de modifier la configuration du node Webhook. Sans ce changement, n8n retourne une erreur au moment de l'execution. Ouvrez le node Webhook et changez le champ Respond :

Champ	Avant	Après
Respond	Last Node	Using Respond to Webhook Node

Une fois ce changement effectue, la chaine complete est :

```
Formatter Agent → HTTP Request → Respond to Webhook
```

Le client curl recoit le JSON d'analyse, webhook.site recoit le rapport en parallele.

Etape 6.4 — Tester

Lancez la requete de test :

```
curl -X POST http://localhost:5678/webhook/hopital \
-H "Content-Type: application/json" \
-d '{"query": "Analyse les patients par service"}'
```

Verifiez deux choses :

1. Le terminal curl recoit bien le JSON d'analyse en reponse
2. L'onglet webhook.site affiche la requete recue avec le contenu JSON complet

Partie 7 — Tests

Lancez le dashboard :

```
python -m http.server 3000
```

Ouvrez <http://localhost:3000/dashboard.html> et testez les cinq requetes suivantes. Pour chacune, notez le JSON retourne par n8n et le resultat affiche dans le dashboard, ainsi que la requete recue sur webhook.site.

1. Analyse les patients par service
2. Quels diagnostics sont les plus critiques ?
3. Quel medecin a la duree de sejour moyenne la plus longue ?
4. Patients encore hospitalises avec leur score de risque
5. Quel service genere le plus de revenus ?

Barème

Partie	Points
Base de donnees creee et verifiee	1 pt
Serveurs MCP lances et fonctionnels	2 pts

Partie	Points
System messages SQL Agent et Analyste	3 pts
System message Alertes Agent et Formatter	2 pts
Bloc A — Comparsateur Agent	3 pts
Bloc B — Fonction get_statistiques_service	3 pts
Bloc C — Envoi webhook.site	2 pts
Qualite des system messages (lisibilite, rigueur)	4 pts
Total	20 pts

Annexe — Fichiers fournis

init_db.sql

```
-- Table services
CREATE TABLE IF NOT EXISTS services (
  id          SERIAL PRIMARY KEY,
  nom         VARCHAR(100) NOT NULL,
  nb_lits     INT NOT NULL,
  responsable VARCHAR(100)
);

-- Table medecins
CREATE TABLE IF NOT EXISTS medecins (
  id          SERIAL PRIMARY KEY,
  nom         VARCHAR(100) NOT NULL,
  prenom      VARCHAR(100) NOT NULL,
  specialite  VARCHAR(100),
  service_id  INT REFERENCES services(id)
);

-- Table patients
CREATE TABLE IF NOT EXISTS patients (
  id          SERIAL PRIMARY KEY,
  nom         VARCHAR(100) NOT NULL,
  prenom      VARCHAR(100) NOT NULL,
  date_naissance DATE NOT NULL,
  sexe        CHAR(1) CHECK (sexe IN ('M', 'F')),
  adresse     TEXT
);

-- Table sejours
CREATE TABLE IF NOT EXISTS sejours (
```

```

    id          SERIAL PRIMARY KEY,
    patient_id  INT REFERENCES patients(id),
    service_id  INT REFERENCES services(id),
    medecin_id  INT REFERENCES medecins(id),
    date_entree DATE NOT NULL,
    date_sortie DATE,
    cout_sejour NUMERIC(10,2),
    statut      VARCHAR(20) DEFAULT 'en_cours' CHECK (statut IN ('en_cours', 'termine',
'urgent'))
);

```

-- Table diagnostics

```

CREATE TABLE IF NOT EXISTS diagnostics (
    id          SERIAL PRIMARY KEY,
    sejour_id   INT REFERENCES sejours(id),
    code_cim    VARCHAR(10),
    libelle     VARCHAR(200) NOT NULL,
    gravite     INT CHECK (gravite BETWEEN 1 AND 5),
    date_pose   DATE NOT NULL
);

```

-- Insertion des services

```

INSERT INTO services (nom, nb_lits, responsable) VALUES
('Cardiologie', 30, 'Dr. Bernard'),
('Neurologie', 25, 'Dr. Moreau'),
('Urgences', 40, 'Dr. Lefevre'),
('Pediatrie', 20, 'Dr. Simon'),
('Chirurgie', 35, 'Dr. Dupont');

```

-- Insertion des medecins

```

INSERT INTO medecins (nom, prenom, specialite, service_id) VALUES
('Bernard', 'Claire', 'Cardiologie', 1),
('Moreau', 'Pierre', 'Neurologie', 2),
('Lefevre', 'Sophie', 'Medecine urgence', 3),
('Simon', 'Marc', 'Pediatrie', 4),
('Dupont', 'Anne', 'Chirurgie', 5),
('Martin', 'Louis', 'Cardiologie', 1),
('Petit', 'Julie', 'Neurologie', 2);

```

-- Insertion des patients

```

INSERT INTO patients (nom, prenom, date_naissance, sexe, adresse) VALUES
('Durand', 'Jean', '1955-03-12', 'M', '12 rue de la Paix, Paris'),
('Leroy', 'Marie', '1978-07-25', 'F', '5 avenue Victor Hugo, Lyon'),
('Morel', 'Paul', '1942-11-08', 'M', '8 boulevard Voltaire, Marseille'),
('Garcia', 'Sophie', '1990-01-30', 'F', '3 rue Gambetta, Toulouse'),
('Martinez', 'Antoine', '1965-09-14', 'M', '21 rue du Commerce, Bordeaux'),
('Robert', 'Helene', '1983-05-22', 'F', '14 impasse des Lilas, Nantes'),
('Bernard', 'Pierre', '1950-12-03', 'M', '7 chemin des Roses, Strasbourg'),
('Thomas', 'Julie', '2001-08-18', 'F', '2 rue Nationale, Lille'),
('Petit', 'Michel', '1938-04-27', 'M', '33 avenue de la Republique, Nice'),
('Roux', 'Isabelle', '1972-10-15', 'F', '9 place du Marche, Rennes'),
('Simon', 'Francois', '1988-02-09', 'M', '18 rue de la Gare, Reims'),
('Laurent', 'Claire', '1960-06-30', 'F', '6 rue Saint-Pierre, Grenoble'),

```

```
( 'Michel',      'Eric',      '1995-11-21', 'M', '11 rue des Fleurs, Toulon'),
( 'Lefevre',     'Nathalie', '1947-08-05', 'F', '25 avenue du General, Angers'),
( 'Adam',        'Thomas',    '2010-03-17', 'M', '4 rue de l Eglise, Le Mans'),
( 'Bonnet',      'Valerie',   '1975-07-11', 'F', '16 rue du Moulin, Aix-en-Provence'),
( 'Fontaine',    'Jacques',   '1932-09-28', 'M', '30 boulevard du Temple, Brest'),
( 'Chevalier',   'Sandrine',  '1992-04-04', 'F', '13 place de la Mairie, Perpignan'),
( 'Rousseau',    'Luc',       '1968-12-19', 'M', '22 rue Victor Hugo, Amiens'),
( 'Vincent',     'Patricia',  '1980-01-07', 'F', '8 impasse du Chateau, Limoges'),
( 'Fournier',    'Remi',      '2005-06-23', 'M', '1 rue de la Fontaine, Clermont-Ferrand'),
( 'Girard',      'Christine', '1955-10-31', 'F', '19 avenue de la Victoire, Metz'),
( 'Mercier',     'Alain',     '1943-03-08', 'M', '27 rue du Faubourg, Nancy'),
( 'Dupuis',      'Laure',     '1998-09-16', 'F', '5 chemin des Vignes, Tours'),
( 'Blanc',       'Georges',   '1970-02-14', 'M', '10 rue de la Republique, Dijon');
```

-- Insertion des sejours

```
INSERT INTO sejours (patient_id, service_id, medecin_id, date_entree, date_sortie,
cout_sejour, statut) VALUES
```

```
(1, 1, 1, '2024-10-01', '2024-10-08', 3200.00, 'termine'),
(2, 2, 2, '2024-10-03', '2024-10-10', 2800.00, 'termine'),
(3, 1, 6, '2024-10-05', NULL, 4100.00, 'en_cours'),
(4, 3, 3, '2024-10-07', '2024-10-08', 950.00, 'termine'),
(5, 5, 5, '2024-10-08', '2024-10-15', 5600.00, 'termine'),
(6, 4, 4, '2024-10-09', '2024-10-14', 1800.00, 'termine'),
(7, 1, 1, '2024-10-10', NULL, 2900.00, 'urgent'),
(8, 4, 4, '2024-10-11', '2024-10-15', 1600.00, 'termine'),
(9, 2, 7, '2024-10-12', NULL, 3700.00, 'en_cours'),
(10, 5, 5, '2024-10-13', '2024-10-20', 6200.00, 'termine'),
(11, 3, 3, '2024-10-14', '2024-10-15', 880.00, 'termine'),
(12, 1, 6, '2024-10-15', NULL, 4500.00, 'urgent'),
(13, 3, 3, '2024-10-16', '2024-10-17', 1100.00, 'termine'),
(14, 2, 2, '2024-10-17', '2024-10-24', 3100.00, 'termine'),
(15, 4, 4, '2024-10-18', '2024-10-21', 1400.00, 'termine'),
(16, 5, 5, '2024-10-19', NULL, 5900.00, 'en_cours'),
(17, 1, 1, '2024-10-20', '2024-10-27', 3600.00, 'termine'),
(18, 3, 3, '2024-10-21', '2024-10-22', 790.00, 'termine'),
(19, 2, 7, '2024-10-22', NULL, 2600.00, 'en_cours'),
(20, 5, 5, '2024-10-23', '2024-10-30', 7100.00, 'termine'),
(21, 4, 4, '2024-10-24', '2024-10-28', 1700.00, 'termine'),
(22, 1, 6, '2024-10-25', NULL, 3300.00, 'urgent'),
(23, 2, 2, '2024-10-26', '2024-11-02', 2900.00, 'termine'),
(24, 3, 3, '2024-10-27', '2024-10-28', 830.00, 'termine'),
(25, 5, 5, '2024-10-28', NULL, 6800.00, 'en_cours');
```

-- Insertion des diagnostics

```
INSERT INTO diagnostics (sejour_id, code_cim, libelle, gravite, date_pose) VALUES
```

```
(1, 'I21.0', 'Infarctus aigu du myocarde', 5, '2024-10-01'),
(2, 'G35', 'Sclerose en plaques', 3, '2024-10-03'),
(3, 'I50.0', 'Insuffisance cardiaque congestive', 4, '2024-10-05'),
(4, 'S06.0', 'Commotion cerebrale', 2, '2024-10-07'),
(5, 'K40.0', 'Hernie inguinale bilaterale', 2, '2024-10-08'),
(6, 'J18.9', 'Pneumonie non precisee', 3, '2024-10-09'),
(7, 'I63.5', 'Infarctus cerebral par occlusion', 5, '2024-10-10'),
(8, 'A09', 'Gastroenterite infectieuse', 1, '2024-10-11'),
```

```
(9, 'G40.0', 'Epilepsie partielle idiopathique', 3, '2024-10-12'),
(10, 'M16.0', 'Coxarthrose primitive bilateral', 2, '2024-10-13'),
(11, 'R07.4', 'Douleur thoracique non specifique', 2, '2024-10-14'),
(12, 'I48', 'Fibrillation et flutter auriculaire', 4, '2024-10-15'),
(13, 'T14.1', 'Plaie de region du corps non precisee', 1, '2024-10-16'),
(14, 'G43.0', 'Migraine sans aura', 2, '2024-10-17'),
(15, 'J06.9', 'Infection aigue des voies respiratoires', 1, '2024-10-18'),
(16, 'C34.1', 'Neoplasme malin du lobe superieur', 5, '2024-10-19'),
(17, 'I10', 'Hypertension essentielle primaire', 3, '2024-10-20'),
(18, 'S52.0', 'Fracture de l olecranon', 2, '2024-10-21'),
(19, 'G61.0', 'Syndrome de Guillain-Barre', 4, '2024-10-22'),
(20, 'K57.3', 'Diverticulose du colon sans peritonite', 3, '2024-10-23'),
(21, 'L03.0', 'Cellulite des doigts et des orteils', 1, '2024-10-24'),
(22, 'I21.4', 'Infarctus du myocarde sous-endocardique', 5, '2024-10-25'),
(23, 'G20', 'Maladie de Parkinson', 3, '2024-10-26'),
(24, 'R55', 'Syncope et collapsus', 2, '2024-10-27'),
(25, 'K80.0', 'Calculs de la vesicule biliaire', 3, '2024-10-28');
```

```
-- Desactivation du Row Level Security
```

```
ALTER TABLE services    DISABLE ROW LEVEL SECURITY;
ALTER TABLE medecins    DISABLE ROW LEVEL SECURITY;
ALTER TABLE patients    DISABLE ROW LEVEL SECURITY;
ALTER TABLE sejours     DISABLE ROW LEVEL SECURITY;
ALTER TABLE diagnostics DISABLE ROW LEVEL SECURITY;
```

mcp_server.py

```
from fastapi import FastAPI
from mcp.server.fastmcp import FastMCP
import psychpg2
import json

# Configuration de la base de donnees
# Modifiez ces parametres selon votre installation PostgreSQL
DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "database": "hopitaldb",
    "user": "postgres",
    "password": "votre_mot_de_passe"
}

mcp = FastMCP("hopital-db")

def get_connection():
    return psychpg2.connect(**DB_CONFIG)

@mcp.tool()
def get_schema() -> str:
    """
```

Retourne le schema complet de la base de donnees hopitaldb.

Appeler cet outil en premier avant toute requete SQL.

"""

```
schema = {
    "tables": {
        "services": {
            "colonnes": ["id", "nom", "nb_lits", "responsable"],
            "description": "Services hospitaliers"
        },
        "medecins": {
            "colonnes": ["id", "nom", "prenom", "specialite", "service_id"],
            "cles_etrangeres": {"service_id": "services.id"},
            "description": "Medecins rattaches a un service"
        },
        "patients": {
            "colonnes": ["id", "nom", "prenom", "date_naissance", "sexe", "adresse"],
            "description": "Patients de l hopital"
        },
        "sejours": {
            "colonnes": ["id", "patient_id", "service_id", "medecin_id", "date_entree",
"date_sortie", "cout_sejour", "statut"],
            "cles_etrangeres": {
                "patient_id": "patients.id",
                "service_id": "services.id",
                "medecin_id": "medecins.id"
            },
            "valeurs_statut": ["en_cours", "termine", "urgent"],
            "description": "Sejours hospitaliers. date_sortie NULL = patient encore
hospitalise"
        },
        "diagnostics": {
            "colonnes": ["id", "sejour_id", "code_cim", "libelle", "gravite",
"date_pose"],
            "cles_etrangeres": {"sejour_id": "sejours.id"},
            "gravite": "entier de 1 (benigne) a 5 (critique)",
            "description": "Diagnostics poses lors d un sejour"
        }
    }
}

return json.dumps(schema, ensure_ascii=False, indent=2)
```

@mcp.tool()

def run_sql(query: str) -> str:

"""

Execute une requete SQL SELECT sur la base hopitaldb et retourne les resultats.

query : requete SQL SELECT a executer

"""

conn = None

try:

conn = get_connection()

cursor = conn.cursor()

```

        cursor.execute(query)
        columns = [desc[0] for desc in cursor.description]
        rows = cursor.fetchall()
        result = [dict(zip(columns, row)) for row in rows]
        return json.dumps(result, ensure_ascii=False, default=str)
    except Exception as e:
        return json.dumps({"error": str(e)})
    finally:
        if conn:
            conn.close()

if __name__ == "__main__":
    import uvicorn
    app = mcp.streamable_http_app()
    uvicorn.run(app, host="127.0.0.1", port=8000)

```

mcp_alertes.py

```

from mcp.server.fastmcp import FastMCP
import json
from datetime import datetime

mcp = FastMCP("hopital-alertes")

# Stockage en memoire des alertes de la session
alertes_session = []

@mcp.tool()
def creer_alerte(niveau: str, service: str, message: str) -> str:
    """
    Cree une alerte medicale et la stocke en memoire pour la session.

    niveau : "info" | "warning" | "critique"
    service : nom du service concerne
    message : description de la situation
    """
    alerte = {
        "id": len(alertes_session) + 1,
        "niveau": niveau,
        "service": service,
        "message": message,
        "horodatage": datetime.now().isoformat()
    }
    alertes_session.append(alerte)
    return json.dumps({
        "statut": "alerte creee",
        "alerte": alerte,
        "total_alertes_session": len(alertes_session)
    }, ensure_ascii=False)

```

```

@mcp.tool()
def calculer_score_risque(age: int, gravite_diagnostic: int, duree_sejour_jours: int) -> str:
    """
    Calcule un score de risque patient sur 100.

    age                : age du patient en annees
    gravite_diagnostic  : gravite du diagnostic principal (1 a 5)
    duree_sejour_jours  : duree du sejour en jours
    """
    score_age          = min(age / 100 * 30, 30)
    score_gravite       = gravite_diagnostic / 5 * 50
    score_duree         = min(duree_sejour_jours / 30 * 20, 20)
    score_total         = round(score_age + score_gravite + score_duree, 1)

    if score_total >= 70:
        niveau = "critique"
    elif score_total >= 40:
        niveau = "modere"
    else:
        niveau = "faible"

    return json.dumps({
        "score":          score_total,
        "niveau_risque":  niveau,
        "detail": {
            "contribution_age":      round(score_age, 1),
            "contribution_gravite":  round(score_gravite, 1),
            "contribution_duree":    round(score_duree, 1)
        }
    }, ensure_ascii=False)

```

```

@mcp.tool()
def envoyer_rapport() -> str:
    """
    Simule l'envoi d'un rapport d'alertes. Retourne le resume des alertes de la session.
    A appeler uniquement si au moins une alerte critique a ete creee.
    """
    nb_critiques = sum(1 for a in alertes_session if a["niveau"] == "critique")
    nb_warnings  = sum(1 for a in alertes_session if a["niveau"] == "warning")
    nb_infos     = sum(1 for a in alertes_session if a["niveau"] == "info")

    rapport = {
        "rapport_envoye":  True,
        "horodatage":      datetime.now().isoformat(),
        "resume": {
            "total":        len(alertes_session),
            "critiques":    nb_critiques,
            "warnings":     nb_warnings,
            "infos":        nb_infos
        }
    }

```

```

    },
    "alertes": alertes_session
}
return json.dumps(rapport, ensure_ascii=False)

if __name__ == "__main__":
    import uvicorn
    app = mcp.streamable_http_app()
    uvicorn.run(app, host="127.0.0.1", port=8001)

```

dashboard.html

```

<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Dashboard Hospitalier</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: Arial, sans-serif;
      background: #f4f6f9;
      color: #333;
      padding: 24px;
    }

    h1 {
      font-size: 1.6rem;
      margin-bottom: 24px;
      color: #1a1a2e;
    }

    .search-bar {
      display: flex;
      gap: 10px;
      margin-bottom: 24px;
    }

    .search-bar input {
      flex: 1;
      padding: 10px 14px;
      border: 1px solid #ccc;
      border-radius: 6px;
      font-size: 1rem;
    }

    .search-bar button {

```

```
padding: 10px 20px;
background: #1a1a2e;
color: white;
border: none;
border-radius: 6px;
cursor: pointer;
font-size: 1rem;
}

.search-bar button:hover {
  background: #2d2d6e;
}

#status {
  margin-bottom: 16px;
  font-size: 0.9rem;
  color: #555;
  min-height: 20px;
}

#summary-box {
  background: white;
  border-radius: 8px;
  padding: 16px 20px;
  margin-bottom: 20px;
  border-left: 4px solid #1a1a2e;
  display: none;
}

#summary-box h2 {
  font-size: 1rem;
  margin-bottom: 8px;
  color: #1a1a2e;
}

.metrics {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
  margin-bottom: 8px;
}

.metric {
  background: #f0f0f8;
  border-radius: 6px;
  padding: 8px 14px;
  font-size: 0.85rem;
}

.metric strong {
  display: block;
  font-size: 1.2rem;
  color: #1a1a2e;
}
```

```

}

.insights-list {
  margin-top: 10px;
  padding-left: 16px;
  font-size: 0.9rem;
}

.alertes-section {
  margin-top: 12px;
}

.alerte {
  padding: 6px 12px;
  border-radius: 4px;
  font-size: 0.85rem;
  margin-top: 6px;
}

.alerte.critique { background: #fdecea; border-left: 3px solid #c0392b; }
.alerte.warning { background: #fff8e1; border-left: 3px solid #f39c12; }
.alerte.info     { background: #e8f4fd; border-left: 3px solid #2980b9; }

.charts-row {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}

.chart-card {
  background: white;
  border-radius: 8px;
  padding: 20px;
}

.chart-card h3 {
  font-size: 0.95rem;
  margin-bottom: 14px;
  color: #1a1a2e;
}

canvas {
  max-height: 260px;
}
</style>
</head>
<body>

<h1>Dashboard Hospitalier</h1>

<div class="search-bar">
  <input type="text" id="query-input" placeholder="Ex : Analyse les patients par
service..." />

```

```

    <button onclick="sendQuery()">Analyser</button>
  </div>

  <div id="status"></div>

  <div id="summary-box">
    <h2 id="summary-text"></h2>
    <div class="metrics" id="metrics-row"></div>
    <ul class="insights-list" id="insights-list"></ul>
    <div class="alertes-section" id="alertes-section"></div>
    <p id="conclusion-text" style="margin-top:10px; font-style:italic; font-size:0.9rem;">
  </p>
  </div>

  <div class="charts-row">
    <div class="chart-card">
      <h3>Patients par service</h3>
      <canvas id="chartService"></canvas>
    </div>
    <div class="chart-card">
      <h3>Diagnostics les plus frequents</h3>
      <canvas id="chartDiag"></canvas>
    </div>
  </div>

  <script>
    const N8N_URL = "http://localhost:5678/webhook/hopital";

    let chartService = null;
    let chartDiag = null;

    async function sendQuery() {
      const query = document.getElementById("query-input").value.trim();
      if (!query) return;

      document.getElementById("status").textContent = "Analyse en cours...";
      document.getElementById("summary-box").style.display = "none";

      try {
        const res = await fetch(N8N_URL, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ query })
        });

        const raw = await res.text();
        const data = JSON.parse(raw);

        document.getElementById("status").textContent = "Analyse terminee.";
        renderDashboard(data);

      } catch (err) {
        document.getElementById("status").textContent = "Erreur : " + err.message;
      }
    }
  </script>

```



```

    },
    options: { responsive: true, plugins: { legend: { display: false } } }
  });

  // Graphique 2 – Diagnostics frequents (doughnut)
  const labelsDiag = (data.charts?.diagnostics_frequents || []).map(d => d.diagnostic);
  const valuesDiag = (data.charts?.diagnostics_frequents || []).map(d => d.total);

  if (chartDiag) chartDiag.destroy();
  chartDiag = new Chart(document.getElementById("chartDiag"), {
    type: "doughnut",
    data: {
      labels: labelsDiag,
      datasets: [{ data: valuesDiag,
        backgroundColor: ["#1a1a2e", "#16213e", "#0f3460", "#533483", "#e94560",
          "#2d6a4f", "#40916c", "#74c69d", "#b7e4c7", "#d8f3dc"] }]}
    },
    options: { responsive: true }
  });
}

// Envoi au clavier (touche Entree)
document.getElementById("query-input").addEventListener("keydown", e => {
  if (e.key === "Enter") sendQuery();
});
</script>

</body>
</html>

```

